JavaScript Code

```
const KEY_TX = 'expense_transactions';
const KEY_CAT = 'expense_categories';
let transactions = JSON.parse(localStorage.getItem(KEY_TX)) || [];
let categories = JSON.parse(localStorage.getItem(KEY_CAT)) ||
  ['Salary', 'Food', 'Travel', 'Shopping', 'Bills', 'Education', 'Health'];

const $ = id => document.getElementById(id);
const fmt = n => '\u20B9' + Number(n).toLocaleString('en-IN', { minimumFractionDigits: 2,
  maximumFractionDigits: 2 });

function save() {
  localStorage.setItem(KEY_TX, JSON.stringify(transactions));
  localStorage.setItem(KEY_CAT, JSON.stringify(categories));
}

/* ---------- Categories ---------- */
function renderCategories() {
  const opts = categories.map(c => `<option>${c}</option>`).join('');
  const cur = $('category').value, fcur = $('fCategory').value;
  $('category').innerHTML = opts;
  $('fCategory').innerHTML = '<option value="all">All Categories</option>' + opts;
  if (categories.includes(cur)) $('category').value = cur;
  if (categories.includes(fcur)) $('fCategory').value = fcur;
}
$('addCatBtn').onclick = () => {
  const name = $('newCategory').value.trim();
  if (!name) return alert('Enter a category name');
  if (categories.some(c => c.toLowerCase() === name.toLowerCase())) return alert('Category
     already exists');
  categories.push(name);
  $('newCategory').value = '';
  save(); renderCategories();
};

/* ---------- Add / Edit ---------- */
$('txForm').onsubmit = e => {
  e.preventDefault();
  const tx = {
    id: $('editId').value || Date.now().toString(),
    type: $('type').value,
    amount: parseFloat($('amount').value),
    date: $('date').value,
    category: $('category').value,
    description: $('description').value.trim()
  };
  if (!(tx.amount > 0)) return alert('Amount must be greater than zero');
  const i = transactions.findIndex(t => t.id === tx.id);
  if (i >= 0) transactions[i] = tx; else transactions.push(tx);
  resetForm(); save(); render();
};
function resetForm() {
  $('txForm').reset(); $('editId').value = '';
  $('formTitle').textContent = 'Add Transaction';
  $('saveBtn').textContent = 'Add';
  $('cancelBtn').classList.add('hidden');
  $('date').value = new Date().toISOString().slice(0, 10);
}
$('cancelBtn').onclick = resetForm;

function editTx(id) {
```

JavaScript Code

```
  const t = transactions.find(x => x.id === id);
  $('editId').value = t.id; $('type').value = t.type; $('amount').value = t.amount;
  $('date').value = t.date; $('category').value = t.category; $('description').value = t.d
    escription;
  $('formTitle').textContent = 'Edit Transaction';
  $('saveBtn').textContent = 'Update';
  $('cancelBtn').classList.remove('hidden');
  window.scrollTo({ top: 0, behavior: 'smooth' });
}
function deleteTx(id) {
  if (!confirm('Delete this transaction?')) return;
  transactions = transactions.filter(t => t.id !== id);
  save(); render();
}

/* ---------- Search & Filter ---------- */
function getFiltered() {
  const q = $('search').value.toLowerCase(), c = $('fCategory').value, ty = $('fType').val
    ue;
  const from = $('fFrom').value, to = $('fTo').value;
  return transactions.filter(t =>
    t.description.toLowerCase().includes(q) &&
    (c === 'all' || t.category === c) &&
    (ty === 'all' || t.type === ty) &&
    (!from || t.date >= from) && (!to || t.date <= to)
  ).sort((a, b) => b.date.localeCompare(a.date));
}
['search', 'fCategory', 'fType', 'fFrom', 'fTo'].forEach(id => $(id).addEventListener('inp
  ut', renderTable));
$('clearBtn').onclick = () => {
  $('search').value = ''; $('fCategory').value = 'all'; $('fType').value = 'all';
  $('fFrom').value = ''; $('fTo').value = ''; renderTable();
};

/* ---------- Rendering ---------- */
function renderTable() {
  const list = getFiltered();
  $('txBody').innerHTML = list.length ? list.map(t => `
    <tr>
      <td>${t.date}</td><td>${t.description}</td><td>${t.category}</td>
      <td><span class="badge b-${t.type}">${t.type}</span></td>
      <td class="amt-${t.type}">${t.type === 'income' ? '+' : '-'}${fmt(t.amount)}</td>
      <td><button class="act" onclick="editTx('${t.id}')">Edit</button>
          <button class="act del" onclick="deleteTx('${t.id}')">Delete</button></td>
    </tr>`).join('') : '<tr><td colspan="6" class="empty">No transactions found</td></tr>'
      ;
}
function renderTotals() {
  const inc = transactions.filter(t => t.type === 'income').reduce((s, t) => s + t.amount,
     0);
  const exp = transactions.filter(t => t.type === 'expense').reduce((s, t) => s + t.amount
    , 0);
  $('totalIncome').textContent = fmt(inc);
  $('totalExpense').textContent = fmt(exp);
  $('balance').textContent = fmt(inc - exp);
}
function monthlyData() {
  const m = {};
  transactions.forEach(t => {
    const k = t.date.slice(0, 7);
```

JavaScript Code

```
    m[k] = m[k] || { income: 0, expense: 0 };
    m[k][t.type] += t.amount;
  });
  return Object.keys(m).sort().map(k => ({ month: k, ...m[k] }));
}
function renderMonthly() {
  const d = monthlyData();
  $('monthBody').innerHTML = d.length ? d.map(r => `
    <tr><td>${r.month}</td><td class="amt-income">${fmt(r.income)}</td>
    <td class="amt-expense">${fmt(r.expense)}</td><td>${fmt(r.income - r.expense)}</td></t
      r>`).join('')
    : '<tr><td colspan="4" class="empty">No data</td></tr>';
}

/* ---------- Chart (Canvas bar chart) ---------- */
function renderChart() {
  const cv = $('chart'), ctx = cv.getContext('2d');
  ctx.clearRect(0, 0, cv.width, cv.height);
  const data = {};
  if ($('chartMode').value === 'category') {
    transactions.filter(t => t.type === 'expense').forEach(t => data[t.category] = (data[t
      .category] || 0) + t.amount);
  } else {
    monthlyData().forEach(r => data[r.month] = r.expense);
  }
  const labels = Object.keys(data), vals = Object.values(data);
  if (!labels.length) { ctx.fillStyle = '#94a3b8'; ctx.font = '16px Arial'; ctx.fillText('
    No expense data to display', 150, 150); return; }
  const max = Math.max(...vals), colors = ['#4f46e5', '#16a34a', '#f59e0b', '#dc2626', '#0
    ea5e9', '#a855f7', '#14b8a6', '#ec4899'];
  const bw = Math.min(60, (cv.width - 60) / labels.length - 14);
  labels.forEach((l, i) => {
    const h = (vals[i] / max) * 200, x = 40 + i * (bw + 14), y = 250 - h;
    ctx.fillStyle = colors[i % colors.length]; ctx.fillRect(x, y, bw, h);
    ctx.fillStyle = '#1e293b'; ctx.font = '11px Arial';
    ctx.fillText(Math.round(vals[i]), x, y - 5);
    ctx.fillText(l.length > 9 ? l.slice(0, 8) + '…' : l, x, 268);
  });
  ctx.strokeStyle = '#94a3b8'; ctx.beginPath(); ctx.moveTo(35, 250); ctx.lineTo(cv.width -
     10, 250); ctx.stroke();
}
$('chartMode').onchange = renderChart;

function render() { renderTable(); renderTotals(); renderMonthly(); renderChart(); }

/* ---------- Init ---------- */
renderCategories();
$('date').value = new Date().toISOString().slice(0, 10);
render();
```